Section 04: Tool Integration

Giving Graphs the Ability to Interact with the World

LangGraph v1.0.x (March 2026)

Mental Model: Tool Integration

The Big Picture

- Tools let graphs **act on the world**: execute code, query databases, call APIs
- Model generates **tool calls**, ToolNode **executes** them, results flow back
- Creates a **feedback loop**: observe -¿ reason → act → observe
- Foundation for agents, task automation, and agentic behavior

Why This Matters

Without tools, LLMs are isolated. Tools unlock real-world impact.

What Are Tools in LangGraph?

Tool Definition

- Callable functions that an LLM can **request** (not execute directly)
- Have a **schema**: name, description, input spec, return type
- Bound to models so they know tools exist: `model.bind_tools()`
- Executed by **ToolNode**, not by the model itself

Tool Ecosystem

- `@tool` decorator: simple functions
- `StructuredTool`: complex schemas with Pydantic
- `ToolNode`: prebuilt executor
- `tools_condition`: prebuilt router

Defining Tools with @tool Decorator

Basic Tool Definition

```python
from langchain_core.tools import tool
@tool
def get_weather(location: str) -> str:
  """Get current temperature in location."""
  return f"Weather in {location}: 72 F "
@tool
def add_numbers(a: int, b: int) -> int:
  """Add two numbers."""
  return a + b
```

Key Points

- Docstring becomes the tool **description**
- Type hints drive schema inference
- Minimal boilerplate, quick prototyping

StructuredTool for Complex Schemas

Pydantic Schema

```python
from pydantic import BaseModel, Field
from langchain_core.tools import StructuredTool
class QueryInput(BaseModel):
  query: str = Field(..., description="Search term")
  limit: int = Field(default=10, description="Max results")
def search_docs(query: str, limit: int) -> str:
  return f"Search: {query}, Limit: {limit}"
tool = StructuredTool.from_function(
  func=search_docs,
  args_schema=QueryInput
)
```

When to Use

StructuredTool when your tool needs complex validation, optional params, or detailed field descriptions.

Binding Tools to Chat Models

model.bind_tools()

```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4")
tools = [get_weather, add_numbers]
bound_model = model.bind_tools(tools)
response = bound_model.invoke(
  "What's the weather in NYC?"
)
print(response.tool_calls)
```

What Binding Does

- Sends tool schemas to model in API call
- Model learns what tools are available
- Model can decide to call tools via `tool_calls`

ToolNode: The Prebuilt Executor

Using ToolNode

```
from langgraph.prebuilt import ToolNode
tool_node = ToolNode(tools=[get_weather, add_numbers])
state = {
  "messages": [
    HumanMessage("What's the weather in NYC?"),
    AIMessage(
      content="",
      tool_calls=[{
        "id": "call_1",
        "name": "get_weather",
        "args": {"location": "NYC"}
      }]
    )
  ]
}
result = tool_node.invoke(state)
```

What ToolNode Does

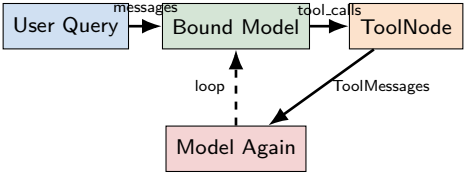
- Executes `tool_calls` from model messages
- Returns `ToolMessage` results back to state

Tool-Calling Loop: How It Works

The Flow

- 1. **User query** arrives at graph
- 2. **Model** (with bound tools) processes message
- 3. **Model generates** `tool_calls` if needed
- 4. **ToolNode** executes the calls
- 5. **ToolMessages** with results added to state
- 6. **Model** processes results and responds
- **Loop** repeats until model stops calling tools

TikZ Diagram: The Tool-Calling Loop



Key Insight

Model never executes tools; it only **requests** them. ToolNode handles execution.

Building a Complete Tool-Calling Graph

Step-by-Step Graph

```python
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition
builder = StateGraph(MessagesState)
builder.add_node("model", model_node)
builder.add_node("tools", ToolNode(tools=[...]))
builder.add_edge(START, "model")
builder.add_conditional_edges(
  "model", tools_condition,
  {"tools": "tools", END: END}
)
builder.add_edge("tools", "model")
graph = builder.compile()
```

tools_condition: The Prebuilt Router

What tools_condition Does

```
from langgraph.prebuilt import tools_condition
def decide_next(state) -> str:
  if state["messages"][-1].tool_calls:
    return "tools"
  else:
    return END
builder.add_conditional_edges(
  "model", tools_condition,
  {"tools": "tools", END: END}
)
```

Why Use tools_condition

Eliminates boilerplate: it already knows how to route based on tool calls.

Handling Tool Errors Gracefully

Common Error Scenarios

- Tool function raises exception
- Model calls nonexistent tool (schema mismatch)
- Tool returns invalid data type

Best Practices

- Wrap tool calls in try-catch
- Return error messages as ToolMessages
- Let model retry with context

Handling Tool Errors: Example

Error Handling Pattern

```
def safe_tool_call(tool_name, args):
  try:
    func = tools_dict[tool_name]
    return func(**args)
  except Exception as e:
    return f"Error: {str(e)}"
def tool_node_with_errors(state):
  for call in state["messages"][-1].tool_calls:
    result = safe_tool_call(
      call["name"], call["args"]
    )
    state["messages"].append(
      ToolMessage(result, tool_call_id=call["id"])
    )
  return state
```

Multiple Tools: Registration and Routing

Multi-Tool Patterns

- Register all tools: `model.bind_tools([tool1, tool2, ...])`
- ToolNode automatically routes by tool name
- Model learns all schemas and picks best tool
- Add tools dynamically at graph creation time

Tool Naming

Use clear, unique names. Model picks tools by matching names to `tool_calls`.

Tool Call Validation and Sanitization

Validation Example

```
from pydantic import validator
class SearchInput(BaseModel):
  query: str = Field(..., min_length=1)
  limit: int = Field(default=10, ge=1, le=100)
  @validator('query')
  def query_not_sql(cls, v):
    if 'DROP' in v.upper():
      raise ValueError("SQL injection attempt")
    return v
```

Always Validate

Never trust model-generated tool args. Use Pydantic validators for safety.

Async Tool Execution

Async Tools

```
import asyncio
@tool
async def fetch_data_async(url: str) -> str:
  """Async fetch from URL."""
  # In real code, use aiohttp
  await asyncio.sleep(0.1)
  return f"Data from {url}"
async def run_graph():
  result = await graph.ainvoke(
    input={"messages": [...]},
    config={...}
  )
```

Performance Boost

Use async tools for I/O-bound operations (API calls, DB queries).

Custom Tool Nodes

When ToolNode Isn't Enough

```
def custom_tool_node(state):
  """Add logging, custom routing, etc."""
  for call in state["messages"][-1].tool_calls:
    print(f"Executing: {call['name']}")
    result = execute_tool(call["name"], call["args"])
    state["messages"].append(
      ToolMessage(result, tool_call_id=call["id"])
    )
  return state
builder.add_node("custom_tools", custom_tool_node)
```

Use Cases

Logging, rate limiting, custom error handling, tool chaining.

Common Pitfalls

Tool Schema Mismatch

- Model calls tool with wrong argument names
- Solution: verify Pydantic schema matches function signature

Forgetting to Bind Tools

- Model has no knowledge of tools if not bound
- Solution: always call `model.bind_tools()` before graph

Misplaced ToolNode

- ToolNode must come **after** model in graph
- Solution: use `tools_condition` for proper routing

Summary: Tool Integration Cheat Sheet

Quick Reference

```python
@tool
def my_tool(x: int) -> int:
  """Tool docstring."""
  return x * 2
model = ChatOpenAI(...).bind_tools([my_tool])
from langgraph.prebuilt import ToolNode, tools_condition
tool_node = ToolNode([my_tool])
builder.add_conditional_edges(
  "model", tools_condition,
  {"tools": "tools", END: END}
)
```

Self-Check: Do You Understand?

1. How does `bind_tools()` change what a model can do?
2. What's the difference between `@tool` and `StructuredTool`?
3. Why doesn't the model execute tools directly?
4. How does `tools_condition` avoid boilerplate?
5. What happens if a tool raises an exception?
6. When should you create a custom tool node?
7. How do you handle multiple tools in one graph?